III) Dijkstra Search (Optimal Solution)

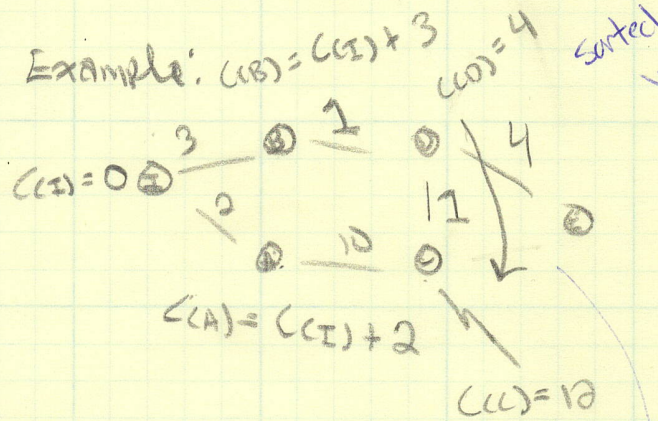
- finds shortest paths on a graph
- Every edge has non-negative cost
- Total cost - sum of edges traversed

A) Priority Queue (Duplicates now need to be resolved)

Cost if you were to traverse path

- Sorted according to cost-to-come [$C(x)$]
- $C^*(x)$: Optimal cost-to-come from initial state to x.
  - Cost is written as $C(x)$ until we know we found min
  - $C(x)$ computed incrementally
  - Cost wavefront instead of step-size wavefront.

Example: $C(B) = C(I) + 3$



Sorted

Node we are on

Reverse Alphabetical

Queue		Order of nodes added
A, B	I	I
B, C	A	B
D, C	B	A
C	D	C
C	E	D
		E

Cost to get to C from D is cheaper that what already exists.

$C(C) = 5$

We would then update the Queue according to new cost of C.

$C(x)$ computed incrementally → Queue is resorted every iteration.

- $C(x') = C^*(x) + l(e_{xx'})$
- x' is unvisited → alive
- x' is alive, look at previous value and adjust sort
- if x is popped from Queue → dead

IV) A-Star Search

Try to reduce the total number of explored states by incorporating a heuristic estimate of the cost to get a given goal.

A) Priority Queue

Cost-to-come: $C(x)$

Cost-to-go: $G(x)$

Estimate cost-to-go: $\hat{G}(x)$ ← Estimate using heuristic

Sort queue according to: $C(x) + \hat{G}(x)$

B) Notes on heuristic

- $\hat{G}(x)$ must be an underestimate
- As $\hat{G}(x) \to G(x)$, fewer vertices visited
- As $\hat{G}(x) \to 0$ A^* degenerates to Djikstra.